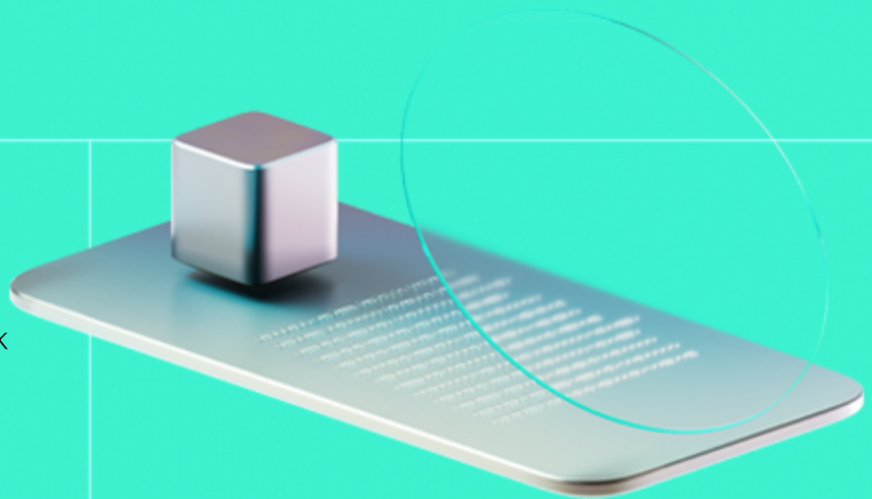




Smart Contract Code Review And Security Analysis Report

Customer: Perceptron Network

Date: 29/07/2026



We express our gratitude to the Perceptron Network team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

Perceptron Network is a decentralised data mesh on Solana that turns idle residential bandwidth and human-contributed data into infrastructure for AI data acquisition and agentic web access, rewarding pseudonymous node operators in the native \$PERC token as a function of measurable per-epoch performance. The audit covered the **PCN Program**, a permissioned bandwidth-reward system on Solana that mints PCN reward tokens to network participants based on bandwidth measurements submitted by a trusted oracle. Emissions are bounded by a scarcity and support curve, by the remaining maximum token supply, and by the SOL support budget deposited for each epoch, and users redeem their allocations through per-epoch claim accounts.

Document

Name	Smart Contract Code Review and Security Analysis Report for Perceptron Network
Audited By	Kerem Solmaz
Approved By	Hamza Sajid
Website	
Changelog	21/07/2026 - Preliminary Report
	29/07/2026 - Final Report
Platform	Solana
Language	Rust
Tags	Claims, Fungible Token, Oracle, Incentives, Centralization
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/Perceptron-Network/pcn-program
Initial Commit	a47dbcd
Final Commit	f79a076

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

6	6	0	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	0
Low	2

Vulnerability	Severity Status	
F-2026-18080 - Missing Access Control On Config Initialization Allows Deployment Hijack And Denial Of Service	Low	Fixed
F-2026-18097 - Unconstrained Refund Recipient In Epoch Finalization Allows Support Budget Refund To Divert From The Original Funder	Low	Fixed
F-2026-18078 - Manual Account Space Calculation Without InitSpace Derivation	Info	Fixed
F-2026-18079 - Passing The Rent Sysvar As An Account Instead Of Using The Syscall Wastes Transaction Space	Info	Fixed
F-2026-18093 - Floating-Point Use In Scarcity Curve Loses Integer Precision For Large Reward Pools	Info	Fixed
F-2026-18096 - Onchain Emission And Reward-Weight Formulas Diverge From The Whitepaper Specification	Info	Fixed

Documentation quality

The project provides a technical whitepaper that describes the network architecture, token economics, and the emission and reward-allocation formulas in detail. The whitepaper is the primary specification, and the onchain program implements a subset of it, namely the per-epoch emission curve and per-node reward settlement. Inline documentation in the program is sparse, and the relationship between the implemented arithmetic and the documented formulas is not recorded in the code. Deviations between the implemented emission and reward-weight formulas and the whitepaper specification were identified and reported.

Code quality

The codebase is organised along Anchor conventions with a clear separation between state definitions, instruction handlers, and reward mathematics, and value-determining accounting is performed in checked integer arithmetic. Account validation relies on Anchor constraints, program derived addresses, and explicit authority checks. Several code-quality and best-practice issues were identified and reported, including manual account space calculation and the use of floating point in value-determining curve arithmetic.

Test coverage

Code coverage of the project is unmeasurable. However, the program ships with unit tests for the reward mathematics and a LiteSVM integration suite exercising configuration authority, epoch validation, the rewards flow, and claim validation. Negative and boundary scenarios are represented, though coverage of the permissionless configuration path and of large-magnitude curve inputs is limited. Adding fuzz tests over the emission curve and reward-weight arithmetic, and explicit tests for unauthorized initialization, is recommended.

Table of Contents

System Overview	6
Privileged Roles:	6
Potential Risks	8
Findings	9
Vulnerability Details	9
Disclaimers	24
Appendix 1. Definitions	25
Severities	25
Potential Risks	25
Appendix 2. Scope	26
Appendix 3. Additional Valuables	28

System Overview

The **PCN Program** is a reward-distribution system that mints a native SPL token to participants in proportion to bandwidth contribution, as measured off chain and reported by a trusted oracle. The program is deployed on Solana under the Anchor framework and is structured around a single global configuration account and a per-epoch lifecycle that moves each epoch from open, to finalized, to swept.

Initialization creates the singleton configuration account through the `initialize_config` instruction. During initialization the program creates a nine-decimal SPL reward mint whose mint authority is a program derived address, a SOL reserve account, and a token reserve vault. The configuration account records the admin and oracle authorities, the reward mint, the reserve addresses and their bumps, the lifetime minted amount, the claim window duration in slots, and the curve parameters that drive emissions.

The epoch lifecycle begins with the `open_epoch` instruction, in which the oracle opens a new epoch and a funder deposits a SOL support budget into the epoch account. After the epoch window ends, the `finalize_epoch` instruction computes the reward pool from the reported total reward weight, the lifetime minted supply, the remaining maximum supply, and the available SOL support. The consumed portion of the support budget is moved into the SOL reserve, the unused portion is refunded, the reward pool is minted into the epoch token vault, and the lifetime minted amount is increased accordingly. The reward pool is the minimum of the scarcity cap, the support cap, and the remaining supply, which keeps issuance bounded on every axis.

Reward allocation is performed by the oracle through the `create_claim` instruction, which creates one claim account per user, deriving its address from the epoch identifier and the user key. Each claim records the user, bandwidth units, quality factor, reward weight, and reward amount, and the cumulative allocated amount is constrained to never exceed the epoch reward pool. Users redeem allocations through the `claim_reward` instruction, which transfers the recorded reward amount from the epoch token vault to a user-owned token account and marks the claim as redeemed so that it cannot be claimed twice. After the claim window expires, the `sweep_epoch` instruction transfers any unclaimed tokens from the epoch token vault into the token reserve vault and marks the epoch as swept.

The system holds value in three places: the per-epoch token vaults that back user claims, the SOL reserve that accumulates consumed support lamports, and the token reserve vault that accumulates swept and unclaimed tokens. All token movements are authorized by the program mint authority program derived address, and all epoch and claim accounts are isolated by seeds that include the epoch identifier, preventing cross-epoch interference.

Privileged Roles:

- **Admin (configuration account):** The admin is a single address set at initialization. The admin can rotate the oracle authority, change the claim window length, and replace the entire emission curve parameter set through the configuration update instruction, subject only to basic validity checks and a constraint that the new maximum supply is not below the amount already minted. Changes take effect immediately with no timelock or multisig. Compromise of the admin key would allow redirecting the oracle authority and reshaping future emissions.

- **Oracle (configuration account):** The oracle is a single address and holds control over the entire reward flow. The oracle opens epochs, finalizes epochs and thereby determines the aggregate reward weight that drives emission and triggers minting, creates each node's claim with its bandwidth and quality inputs, and sweeps residual balances. The per-node performance score is computed entirely from the two scalars the oracle supplies, so the oracle effectively decides how much every node receives. Compromise or misbehaviour of the oracle key would allow inflating or suppressing emissions within the curve bounds and misallocating rewards among nodes.
- **Mint authority (program derived address):** The mint authority is a program derived address with no external key. It signs the reward token minting during finalization and the token transfers during claims and sweeps. Its powers are exercised only through the program instructions and cannot be invoked directly by any external signer.
- **Configuration initializer (initialization instruction):** The initialization instruction that creates the singleton configuration account is not access-controlled and can be called by any account, which sets the admin and oracle authorities and binds the reward mint. This is documented in the findings.

Potential Risks

- **Scope Definition and Security Guarantees:** The audit covers the pcn-program only. The Perceptron system described in the whitepaper depends on a substantial offchain infrastructure, including the residential node clients, the performance-metric collection and verification pipeline, and the oracle service that reports aggregate work and per-node scores. The Data Questing bounty distribution, the protocol funding and claim burns, staking, and governance described in the whitepaper are not present in the audited program. These offchain components and unimplemented mechanisms were not assessed, and their correctness is presumed; flaws in the oracle or metric pipeline would directly affect the integrity of onchain emissions and reward allocation regardless of the program's correctness.
- **Centralized Oracles as Data Sources:** The program derives every value-determining quantity from inputs supplied by a single oracle authority. The aggregate reward weight passed to epoch finalization sets the size of the minted reward pool, and the bandwidth and quality inputs passed to claim creation set each node's share. The program applies only range and consistency checks and does not reconstruct or constrain the documented multi-metric performance score, so the accuracy of emissions and reward distribution rests entirely on the honesty and correctness of this single offchain data source. An incorrect or manipulated oracle report would misprice emissions or misallocate rewards among nodes without violating any onchain invariant.
- **Administrative Key Control Risks:** The admin and oracle authorities are each a single address with no multisig or key-separation. The admin can rotate the oracle and rewrite the emission curve parameters, and the oracle controls the full epoch and claim lifecycle. Concentrating the emission-shaping capability and the reward-reporting capability in single keys means that compromise of either key grants control over the reward flow, and the two roles may in practice be held by related parties.
- **Absence of Time-lock Mechanisms for Critical Operations:** The configuration update instruction applies changes to the oracle authority, the claim window, and the emission curve parameters immediately upon a valid admin call. There is no delay, announcement period, or review window before a new curve or a new oracle takes effect, so node operators have no opportunity to observe or react to a change in emission economics or reporting authority before it governs live epochs.
- **Missing Access Control on Deployment Initialization:** The instruction that creates the singleton configuration account enforces no access control and can be executed by any account before the legitimate deployer. The first caller sets the admin and oracle authorities to arbitrary addresses and binds the configuration to a caller-controlled mint, after which the deterministic configuration address cannot be reinitialized. This creates a denial-of-service and identity-hijack risk on deployment and is reported as a finding; deployment should bind initialization to the intended authority atomically or through a trusted deployment procedure.

Findings

Vulnerability Details

F-2026-18080 - Missing Access Control On Config Initialization Allows Deployment Hijack And Denial Of Service - Low

Description:

The `initialize_config` instruction creates the singleton `config` account that governs the entire reward system. The `config` account is initialized at the fixed program derived address derived from the `config` seed, so it can be created exactly once for a given program identifier. The instruction sets the `admin` and `oracle` authorities, the reward mint, and the curve parameters that every later instruction relies on.

The instruction enforces no access control over who may call it. The only signer in the accounts struct is `payer`, and that signer is not compared against the program upgrade authority or any other trusted identity. The privileged `admin` and `oracle` addresses are taken directly from the caller supplied arguments, and the `reward_mint` is created from a keypair the caller provides, so its mint authority becomes the program mint authority PDA under a mint the caller controls.

Because the address is deterministic and the instruction is permissionless, any observer can submit `initialize_config` before the legitimate deployer does. The first successful caller sets the admin and oracle authorities to arbitrary addresses and binds config to a caller-chosen mint. Once the `config` account exists, every later `initialize_config` attempt fails because the account is already allocated, so the legitimate deployer can no longer initialize the program at that identifier and is forced to redeploy under a new program identifier.

The impact is bounded because the reward mint is created during initialization rather than referencing a pre-existing token, so no already-valuable asset is stolen. The concrete damage is a denial of service on deployment and a hijack of the program identity: any off-chain infrastructure that trusts the `config` account of the known program identifier would resolve to an attacker controlled `oracle` and `mint`, and the attacker holds full control of the reward flow, including opening and finalizing epochs, creating claims, and updating curve parameters.

Assets:

- `./src/instructions/config/initialize_config.rs`
[<https://github.com/Perceptron-Network/pcn-program>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation:

- Restrict `initialize_config` to a trusted identity, such as the program upgrade authority.
- Add the program data account to the accounts struct and constrain the caller against its upgrade authority.

Sample fix:

```
#[derive(Accounts)]
pub struct InitializeConfig<'info> {
    #[account(mut)]
    pub payer: Signer<'info>,
    // Other Code
    #[account(
        constraint = program_data.upgrade_authority_address == Some(payer.key())
        @ PcnError::UnauthorizedAdmin
    )]
    pub program_data: Account<'info, ProgramData>,
    #[account(constraint = program.programdata_address()? == Some(program_data.key()))]
    pub program: Program<'info, crate::program::PcnProgram>,
    // Other Code
}
```

Resolution:

Fixed in commit `6d59562`.

`initialize_config` now binds the caller to the program upgrade authority: it requires a `program` / `program_data` pair whose `programdata_address` matches, and constrains `program_data.upgrade_authority_address == Some(payer.key())` with `PcnError::UnauthorizedInitializer`. Only the deployer's upgrade authority can

create the singleton config, closing the hijack and deployment DoS window.

F-2026-18097 - Unconstrained Refund Recipient In Epoch Finalization Allows Support Budget Refund To Divert From The Original Funder - Low

Description:

The `open_epoch` instruction takes a funder signer that is distinct from the oracle and transfers `support_budget_lamports` into the epoch PDA. The `finalize_epoch` instruction later returns the unconsumed part of that budget to a `refund_target` account:

```
#[derive(Accounts)]
pub struct FinalizeEpoch<'info> {
    pub oracle: Signer<'info>,
    /// CHECK: Receives unused support lamports.
    #[account(mut)]
    pub refund_target: UncheckedAccount<'info>,
    // Other Code
}
```

The account carries no address constraint, no `has_one` relation, and no seed derivation, and the only authorization in the instruction is the oracle signer check. The refund recipient is therefore chosen freely by the oracle at finalization time. The `Epoch` account does not store the funder pubkey, so the program has no record against which a recipient could be validated.

The funder has no onchain guarantee of receiving its own refund. The oracle is a trusted role, so the exposure is limited to the unconsumed budget of an epoch, but a compromised or misconfigured oracle can route every refund to an address it controls.

Assets:

- `./src/instructions/epoch/finalize_epoch.rs` [<https://github.com/Perceptron-Network/pcn-program>]
- `./src/instructions/epoch/open_epoch.rs` [<https://github.com/Perceptron-Network/pcn-program>]
- `./src/state/epoch.rs` [<https://github.com/Perceptron-Network/pcn-program>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation:

- Store the funder pubkey on the `Epoch` account in `open_epoch` and enforce the refund recipient against it in `finalize_epoch`, for example with `require_keys_eq!` or a `has_one` constraint.
- If a configurable destination is intended instead, pin it to an address stored in the config account alongside `sol_reserve` so the oracle cannot choose it per call.

Resolution:

Fixed in commit `f79a076`.

`open_epoch` records the funding signer as the immutable `epoch.support_funder`, and `finalize_epoch` binds the refund recipient with `address = epoch.support_funder @ PcnError::InvalidRefundTarget`, so the unused support budget can only be credited to the original funder.

F-2026-18078 - Manual Account Space Calculation Without InitSpace Derivation - Info

Description:

The program stores three account types, `Claim`, `Config`, and `Epoch`, and derives the space reserved at account creation from a manual constant. Each type defines its length as `LEN` using `8 + size_of::<Self>()`, and the corresponding `init` constraints allocate the account with that value.

Anchor's preferred approach is to derive `InitSpace` and allocate with `8 + Type::INIT_SPACE`. That sizes accounts to the exact Borsh layout and avoids paying rent for compiler alignment padding that is never written.

This is a best-practice / code-quality note, not a functional defect in the current code.

Assets:

- `./src/state/config.rs` [<https://github.com/Perceptron-Network/pcn-program>]
- `./src/state/epoch.rs` [<https://github.com/Perceptron-Network/pcn-program>]
- `./src/state/claim.rs` [<https://github.com/Perceptron-Network/pcn-program>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

- Derive `InitSpace` on `Claim`, `Config`, and `Epoch`, and nested types such as `CurveParams` and `EpochStatus`.
- Replace `LEN` with `8 + Type::INIT_SPACE` in each `init` constraint's `space` argument.

Resolution:

Fixed in commit [6d59562](#).

`Config`, `Epoch`, and `Claim` now derive `InitSpace` and define `LEN` as `8 + Self::INIT_SPACE` instead of `8 + size_of::<Self>()`, so allocation matches the exact Borsh layout rather than compiler-padded struct size. `CurveParams` and `EpochStatus` also derive `InitSpace` so the nested layout is sized precisely. Each state file adds a unit test asserting that the serialized length equals `INIT_SPACE`.

[F-2026-18079](#) - Passing The Rent Sysvar As An Account Instead Of Using The Syscall Wastes Transaction Space - Info

Description:

The `initialize_config` and `open_epoch` instructions each declare the rent sysvar as an explicit account through `rent: Sysvar<'info, Rent>` in their accounts struct. This requires the caller to pass the rent sysvar account in every invocation of these instructions, and the account occupies one of the limited account slots and adds to the serialized transaction size.

The rent parameters are available on chain through the `Rent::get` syscall, which reads the sysvar directly from the runtime without the account being provided in the transaction. Requesting the sysvar as an account is a legacy pattern that the runtime no longer requires. The current code does not read from the `rent` account in the instruction bodies at all; the field is present only to satisfy the older account-based access model, so it provides no functional benefit while still consuming an account slot and instruction data.

Replacing the account with the syscall reduces the account count and transaction size for both instructions and removes a dependency the caller must otherwise supply correctly. There is no security impact; the finding is a resource and clarity improvement.

Assets:

- `./src/instructions/epoch/open_epoch.rs` [<https://github.com/Perceptron-Network/pcn-program>]
- `./src/instructions/config/initialize_config.rs` [<https://github.com/Perceptron-Network/pcn-program>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

- Remove the `rent: Sysvar<'info, Rent>` field from the `InitializeConfig` and `OpenEpoch` accounts structs.
- Where the rent value is needed inside an instruction, obtain it with `Rent::get()?` rather than from a passed account.

Resolution:

Fixed in commit `6d59562`.

The `rent: Sysvar<'info, Rent>` field was removed from both the `InitializeConfig` and `OpenEpoch` accounts structs, so neither instruction requires the caller to pass the rent sysvar account.

[F-2026-18093](#) - Floating-Point Use In Scarcity Curve Loses Integer Precision For Large Reward Pools - Info

Description:

At epoch finalization the `finalize_epoch` instruction bounds the amount of reward tokens minted by a scarcity cap derived from the emission curve. The cap is produced by the `calculate_scarcity_cap` function from the reported total reward weight, the lifetime minted amount, and the configured curve parameters, denominated in token base units at nine decimals. The curve is `max_epoch_mint * (1 - exp(-total_reward_weight / saturation_units)) / (1 + lifetime_curve_minted_amount / history_minted)`, and it is evaluated in double-precision floating point:

```
let reward_weight_ratio = total_reward_weight as f64 / curve.saturation_units
as f64;
let minted_history_ratio = lifetime_curve_minted_amount as f64 / curve.history_minted
as f64;
let scarcity_base_units = curve.max_epoch_mint as f64 * (1.0 - libm::exp(-reward_weight_ratio))
/ (1.0 + minted_history_ratio);
round_curve_base_units(scarcity_base_units)
```

Floating point is a poor fit for value-determining arithmetic on Solana for two independent reasons. The eBPF target has no native floating-point instructions, so every `f64` multiply, divide, and the `libm::exp` call is lowered to software-emulated routines. This costs substantially more compute units than the equivalent integer arithmetic and consumes budget that the rest of the transaction could use, while the surrounding accounting is already performed in checked `u128` integer math. Independently, floating point loses precision as magnitudes grow: an IEEE 754 double represents consecutive integers exactly only up to 2^{53} , which is `9_007_199_254_740_992` base units, or about `9_007_199` whole tokens. Any scarcity result above that magnitude cannot be represented exactly, and the computed cap drifts from the exact curve by a rounding error that scales with the size of the result.

The drift is a consistent underestimate of the specified curve. With `max_epoch_mint` set to one billion whole tokens (`1e18` base units) and a weight ratio of `0.3`:

<code>max_epoch_mint</code> (base units)	computed cap	exact curve value	shortfall
<code>1_000_000_000 * 1e9</code>	<code>25918177931828211</code>	<code>25918177931828213</code>	22 base units
	2	4	

max_epoch_mint (base units)	computed cap	exact curve value	shortfall
4_000_000_000 * 1e9	10367271172731284	10367271172731285	88 base units
	48	36	

The deviation does not create a consensus or determinism risk. The eBPF softfloat routines and `libm::exp` are deterministic, so every validator computes the identical value. The scarcity result is further clamped downstream by the support cap and the remaining supply through a minimum and never exceeds `max_epoch_mint`, so the error cannot inflate issuance beyond the curve. The magnitude of the drift is on the order of tens of base units, far below one whole token, so no meaningful value is misallocated. The issue is a correctness, robustness, and compute-cost concern: the on-chain emission cap does not exactly match the specified formula for large pools, and floating point is a fragile and compute-expensive foundation for value-determining arithmetic on this target.

Assets:

- `./src/rewards.rs` [<https://github.com/Perceptron-Network/pcn-program>]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	2/5
Exploitability:	Semi-Dependent
Complexity:	Complex
Severity:	Info

Recommendations

Remediation:

- Evaluate the scarcity curve in deterministic unsigned integer fixed-point rather than floating point. Choose a fixed-point scale, carry every intermediate in `u128`, and reserve approximation for the exponential alone so the multiply and divide stay exact. This removes the software-float compute overhead and keeps the result within one base unit of the exact curve across all `u64` inputs. Reference tables of fixed-point exponential constants or an existing audited fixed-point math library can supply these values directly.
- If the floating-point implementation is retained for the audited version, document the residual precision behavior against the specification so the emission-cap deviation for large pools is a recorded and accepted

property rather than an implicit one. Use checked arithmetic over that part of the code.

Resolution:

Fixed in commit [6d59562](#).

`calculate_scarcity_cap` no longer uses `f64` or `libm::exp`; the curve is evaluated entirely in Q64.64 fixed-point integer math over `u128`. The exponential is computed by `exp_neg_q64` via a positive-term Taylor series with reciprocal and range reduction, and ratios and scaling use checked integer helpers (`ratio_to_q64`, `q64_mul`, `q64_scale_u64_round`), removing both the software-emulated float cost and the `2^53` precision ceiling.

[F-2026-18096](#) - Onchain Emission And Reward-Weight Formulas

Diverge From The Whitepaper Specification - Info

Description:

The program implements the two core reward formulas from the whitepaper token economics, but the implemented arithmetic diverges from the specification in two respects. Both concern value-determining code that is present and executed onchain, so the deviation is between what the code computes and what the documentation states it should compute. Neither deviation is exploitable and neither corrupts the internal accounting of the program; the concern is correctness against the specification.

The first divergence is the absence of the per-epoch emission multiplier. Section 4.2 defines the per-epoch mint as $\text{mint} = R * M * (1 - e^{-(w/W)}) / (1 + h/H)$, where:

R is an emission multiplier in the range $0 < R \leq 1$, set per epoch by the protocol.

M , w , W , h , and H are the maximum per-epoch mint, epoch work, saturation factor, lifetime emitted amount, and difficulty factor.

The `calculate_scarcity_cap` function implements every term except R :

```
let reward_weight_ratio = total_reward_weight as f64 / curve.saturation_units
as f64;
let minted_history_ratio = lifetime_curve_minted_amount as f64 / curve.history_minted as f64;
let scarcity_base_units = curve.max_epoch_mint as f64 * (1.0 - libm::exp(-reward_weight_ratio))
/ (1.0 + minted_history_ratio);
```

This evaluates the whitepaper formula with R fixed at 1 . Neither `CurveParams` nor `FinalizeEpochArgs` carries a per-epoch multiplier, and no code path scales the computed cap, so the protocol has no onchain lever to throttle emissions below the activity-and-scarcity maximum for a given epoch. The documented ability to dampen emissions per epoch, for example during bootstrapping or in response to abnormal reported work, is not present in the implementation.

The second divergence is that the per-node reward weight is a two-factor product rather than the documented multi-metric performance score.

Section 4.3 defines node allocation as $\text{Reward}_i = \text{mint} * (P_i / \text{sum}(P))$, where the performance score is a governance-weighted combination of four metrics:

```
P_i = w1*Uptime_i + w2*Bandwidth_i + w3*FulfilmentRate_i + w4*QuestScore_i
```

```
w1 + w2 + w3 + w4 = 1
```

The `compute_reward_weight` function instead computes the weight as a single product of two oracle-supplied scalars:

```
let weight = u128::from(bandwidth_units)
    .checked_mul(u128::from(quality_factor_ppm))
    .ok_or(PcnError::MathOverflow)?
    .checked_div(u128::from(QUALITY_PPM_SCALE))
    .ok_or(PcnError::MathOverflow)?;
```

The proportional allocation structure matches the specification, since `compute_claim_amount` distributes `reward_pool * reward_weight / total_reward_weight`, corresponding to `mint * P_i / sum(P)`. The onchain per-node score is defined as `bandwidth_units * quality_factor_ppm`, not the four-term weighted sum. The four documented metrics and the governance weight vector are not represented in the program. The scoring computation is delegated to the offchain oracle, which reports the two scalars `bandwidth_units` and `quality_factor_ppm` per claim through the `create_claim` instruction, and the onchain formula neither reconstructs nor constrains the documented composite. The two onchain fields do not correspond to the documented score composition, and the applied performance model cannot be verified from the code.

The overall impact is limited to specification correctness: the emission cap omits a documented control parameter, and the reward weight uses a formula different from the documented performance score, shifting the score derivation entirely to the trusted oracle.

Assets:

- `./src/rewards.rs` [<https://github.com/Perceptron-Network/pcn-program>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	3/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

- If a per-epoch emission multiplier is intended for this version, add it as an explicit parameter, for example a parts-per-million multiplier on `FinalizeEpochArgs` or `CurveParams`, validated to $0 < R \leq 1$, and apply it to the computed scarcity cap. If a multiplier fixed at `1` is the intended behavior for this version, document that the whitepaper `R` is fixed at one onchain so the omission is a recorded and accepted property rather than an implicit one.
- Document that the onchain `reward_weight` is defined as `bandwidth_units * quality_factor_ppm` and that the whitepaper multi-metric performance score is composed offchain by the oracle before being encoded into these two fields. If any part of the documented composite is intended to be enforced or reproduced onchain, implement it explicitly; otherwise record the reliance on the oracle for the full score derivation as an accepted trust assumption.

Resolution:

Fixed in commit `4e087f6`.

Both divergences are resolved, each through the stronger of the two options offered in the recommendation.

The emission multiplier `R` is now an explicit curve parameter rather than an implicit constant. It is validated to the documented range on every use and applied to the scarcity cap, so the on-chain emission curve carries the same terms as Section 4.2. The exponential was also moved from floating point to fixed point in the same change.

The reward weight is no longer an opaque product of two oracle-supplied scalars. The oracle now submits the four normalized performance metrics per claim, each bounded on-chain, and the program itself computes the Section 4.3 composite using governance weights that are validated to sum to one. Those weights are frozen onto the epoch when it opens, so changing them mid-epoch cannot re-rank work that has already been performed. The README records the resulting model, including that the aggregate epoch weight used as the allocation denominator is still supplied by the oracle at finalization.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/Perceptron-Network/pcn-program
Initial Commit	a47dbcdade999faa5b0703a1d30711c73e58b8dcf
Final Commit	f79a0761bc008cd4195cea7de3d48f37f99610de
Whitepaper	https://docs.google.com/document/d/1WdGkGWHMy32vILSBYGTyAco78nH5Wtk-ofszt1vnbqg
Requirements	README.md
Technical Requirements	README.md

Asset	Type
./src/constants.rs [https://github.com/Perceptron-Network/pcn-program]	Smart Contract
./src/error.rs [https://github.com/Perceptron-Network/pcn-program]	Smart Contract
./src/instructions/claim/claim_reward.rs [https://github.com/Perceptron-Network/pcn-program]	Smart Contract
./src/instructions/claim/create_claim.rs [https://github.com/Perceptron-Network/pcn-program]	Smart Contract
./src/instructions/claim/mod.rs [https://github.com/Perceptron-Network/pcn-program]	Smart Contract
./src/instructions/config/initialize_config.rs [https://github.com/Perceptron-Network/pcn-program]	Smart Contract
./src/instructions/config/mod.rs [https://github.com/Perceptron-Network/pcn-program]	Smart Contract
./src/instructions/config/update_config.rs [https://github.com/Perceptron-Network/pcn-program]	Smart Contract
./src/instructions/epoch/finalize_epoch.rs [https://github.com/Perceptron-Network/pcn-program]	Smart Contract
./src/instructions/epoch/mod.rs [https://github.com/Perceptron-Network/pcn-program]	Smart Contract
./src/instructions/epoch/open_epoch.rs [https://github.com/Perceptron-Network/pcn-program]	Smart Contract
./src/instructions/epoch/sweep_epoch.rs [https://github.com/Perceptron-Network/pcn-program]	Smart Contract
./src/lib.rs [https://github.com/Perceptron-Network/pcn-program]	Smart Contract

Asset	Type
./src/rewards.rs [https://github.com/Perceptron-Network/pcn-program]	Smart Contract
./src/state/claim.rs [https://github.com/Perceptron-Network/pcn-program]	Smart Contract
./src/state/config.rs [https://github.com/Perceptron-Network/pcn-program]	Smart Contract
./src/state/epoch.rs [https://github.com/Perceptron-Network/pcn-program]	Smart Contract
./src/state/mod.rs [https://github.com/Perceptron-Network/pcn-program]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.